

No Way Out

No Way Out (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): *No Way Out* is a **2-player top-down brawler** where two scavengers enter **shifting labyrinths that mutate unpredictably**, racing to collect loot while **sabotaging each other's paths**. A **real-time C++ simulation engine** tracks wall directions, collision states and dynamic map shifts every frame so transitions are seamless — no softlocks, no desyncs — and a **robust LAN networking layer** keeps both players synchronized through every mutation and trap trigger. Named load-bearing C++: the game loop **remembers all wall directions each frame** then randomises/changes a chosen wall, aiming for a **stable step for up to 4 players** (targeting 2), running **deterministically**; and a **real-time physics/collision engine** that keeps wall collision correct as walls change direction. Named technical features: an **algorithm that shifts the layout while guaranteeing a path back to spawn** (no softlocking); **clean, fast tile updates** so players aren't visibly affected by layout changes; **robust LAN networking** that resists desync through layout changes; and the **collision engine** for direction-changing walls. Tool: a **custom level editor** for building maps (environment + traps). Design features: dynamic labyrinth mutation forcing constant adaptation; a **loot system** with limited carry capacity and deposit scoring; **sabotage mechanics** (traps, path-blocking, altering the labyrinth); and a **timed survival challenge** under a strict countdown. Monetization is a **cosmetic-only shop** (no gameplay effect). Target platform: **Native Android mobile, tested on Google Pixel 9a**.

Note (module completeness): the proposal doesn't mention an **External Database** or accounts, but the module mandates Firebase as the external DB — plan to persist **cosmetic ownership / unlock records** (and optionally match results) there (a standard, small addition; see the External Database row + M1G12-style integration). Not a category concern; just build it in.

References:

- [Ultimate Chicken Horse on Steam](#) (a competitive party game where players place traps and hazards to sabotage each other while racing an obstacle course — the place-traps / sabotage-the-opponent precedent; also ships a level editor)
- [SpeedRunners on Steam](#) (a competitive multiplayer race where players use items to sabotage rivals and the last one on screen is eliminated — the race-under-pressure + sabotage precedent)
- [Labyrinth \(Ravensburger board game\) — Wikipedia](#) (the classic shifting-tile maze: players slide rows of tiles to reshape the labyrinth every turn, opening and closing paths — the mutating-maze precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	No Way Out
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	✓

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — C++ engine via JNI rendering through OpenGL ES in a GLSurfaceView, Kotlin app shell; device: Pixel 9a) — a real-time top-down brawler with touch controls; the Native PC build is the development/tools vehicle (the level editor). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for cosmetic-ownership persistence (and optional match records); LAN peer-to-peer is the in-match transport.

Front end: A 2D top-down renderer — the mutating labyrinth grid, both scavengers, loot, traps, the countdown/score HUD and the deposit/carry indicators, updated seamlessly as walls shift. **Languages:** Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the realtime 2D in a GLSurfaceView; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: The deterministic real-time C++ simulation loop (per-frame wall-direction state, the maze-mutation algorithm with a guaranteed path-back-to-spawn, collision resolution for direction-changing walls, loot/sabotage/trap logic and the countdown), plus the LAN networking layer keeping both clients in sync through every mutation; Firebase for cosmetic ownership on the Extended tier. **Languages:** C++ (engine + networking); Firebase-backed services on the Extended tier.

Backend scope: Local + LAN now, server-ready (T2) — the cosmetic-persistence seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — each frame it records every wall's direction, mutates the chosen wall(s), re-resolves collision so a turning wall never traps or tunnels a player, advances loot/sabotage/traps and the countdown, and holds a stable deterministic step (targeting 2 players, architected for up to 4).
- *Networking:* **core pillar** — a robust **LAN** layer (peer/host) keeps both clients synchronized through every maze mutation and trap trigger; determinism + a shared seed for the mutation stream keeps the two simulations identical, with desync detection so a divergence is caught, not silently played out.
- *Data-driven:* tile/wall types, trap definitions, loot values + carry/deposit rules, the mutation parameters (seed, cadence, constraints) and the cosmetic catalogue are all data — a new trap, tile or cosmetic is a data change (authored via the level editor).
- *2D:* the top-down maze renderer, the shifting-wall transitions, loot/trap sprites and the countdown/score HUD (custom-rendered). *3D:* not needed.
- *Custom tool:* the **level editor** — place environment + traps, define spawn/loot/deposit points and the mutable wall regions, with round-trip JSON so designers build maps without code.

- *External DB*: cosmetic ownership / unlock records (Firebase — add this per the module). *Auth* »: optional accounts. *Analytics* »: optional — match length + sabotage/loot stats for balancing.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in `milestone-goals-pool-per-category.pdf` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the deterministic per-frame maze/loot/countdown loop + touch input.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a deterministic fixed-timestep C++ + loop that records every wall's direction each frame, mutates the chosen wall(s), advances loot/sabotage/traps and the countdown, decoupled from rendering and ready for LAN lockstep; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the maze generator (M1G7), collision (M1G5) and networking (M1G11) run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Entity/component model with tile/wall types, trap definitions, loot values + carry/deposit rules, the mutation parameters (seed, cadence, constraints) and the cosmetic catalogue in hot-reloadable JSON — a new trap/tile/cosmetic is a data change.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (the level editor), M1G7 (maze data), M1G5 (collision shapes from tile data).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays (wall-direction state, path-to-spawn validation, LAN frame/input buffer), a per-frame state-hash desync detector across the two clients, Tracy profiling of a full mutation + trap-heavy

match to hold a stable rate on Android, and Catch2 tests (deterministic replay, "always a path back to spawn"), on a GitHub Actions Windows+Android matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the desync detection M2G7 hardens.

- **G5 Physics & Collision** — the real-time collision engine that stays correct as walls change direction.

Tech & dependencies

- **Tech:** A C++ collision system (uniform-grid broadphase over the tile map + resolution) that re-derives wall colliders the instant a wall mutates, so a turning wall never tunnels or traps a player; GLM math, deterministic and identical on both clients.
- **Prerequisites:** G1 (loop & tick), G2 (tile/wall data).
- **Feeds into:** M2G11 (advanced/continuous collision); the "seamless transitions, no softlocks" pillar.

- **G7 Procedural Level Generation** — the **maze-mutation algorithm with a guaranteed path back to spawn**.

Tech & dependencies

- **Tech:** Seeded RNG + a graph/connectivity check (union-find or BFS/DFS reachability) that shifts the layout each cadence while validating that both scavengers retain a path to spawn/deposit — a mutation that would softlock is rejected or repaired; validation tests prove the invariant.
- **Prerequisites:** G2 (maze/tile data model).
- **Feeds into:** M2G14 (labyrinth variety/pacing); the replayability + "no softlock" pillars.

- **G11 Networking Foundation** — the robust **LAN** layer that resists desync through mutations.

Tech & dependencies

- **Tech: Web build:** LAN sockets (Winsock2/BSD, or ENet/SDL_net) with a lockstep/shared-seed protocol so both clients run the identical mutation stream, an input/frame buffer, and lobby/host discovery; serialization + a state-hash desync guard. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G1 (deterministic loop), G3 (desync detection).
- **Feeds into:** M2G7 (match sync & prediction hardening); the LAN-sync pillar.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — on-device validation + the cosmetic-persistence seam.

Tech & dependencies

- **Tech: Web build:** Memory pools for tiles/traps/loot, Tracy profiling of a mutation- and trap-heavy match, ASan on MSVC, and on-device validation (Pixel 9a) holding the frame budget; the Firebase/Linux-server seam (cosmetic ownership) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable + a stable rate under heavy mutation.

- **G2 ★ Content Editor — Core (Level Editor)** — build maps: environment, traps, spawn/loot/deposit points and mutable wall regions.

Tech & dependencies

- **Tech:** Dear ImGui + the shared renderer: paint tiles/walls, place traps and loot/deposit/spawn points, mark the regions that may mutate, with round-trip JSON serialization, live preview and undo/redo.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (preview).
- **Prerequisites:** G2 (tile/trap/loot data), M1G5 (collision shapes preview).
- **Feeds into:** All map content; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android Debug builds, tests (deterministic replay, path-to-spawn invariant, LAN-sync under simulated latency/loss) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES

drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

• **G7 Networking — Match Sync & Prediction (LAN Hardening)** — the LAN layer hardened against packet loss/jitter while walls mutate.

Tech & dependencies

- **Tech:** Input-delay/lockstep tuning, handling of dropped/late packets (stall + catch-up), and per-frame state-hash desync detection + recovery so a mid-mutation divergence is caught and healed; validated under simulated packet-loss/jitter/lag on LAN.
- **Prerequisites:** M1G11 (networking foundation), M2G1 (optimization).
- **Feeds into:** A desync-free 2-player match at showcase; the LAN pillar.

• **G11 Physics — Advanced** — continuous collision so fast play + shifting walls never tunnel.

Tech & dependencies

- **Tech:** Continuous collision detection (swept tests) and constraint/material handling so a player moving at speed against a just-mutated wall resolves correctly every frame; GLM math, deterministic on both clients.
- **Prerequisites:** M1G5 (physics & collision).
- **Feeds into:** The "seamless, no-softlock" transitions under real play speed.

• **G14 Procedural Generation II** — labyrinth variety + mutation pacing for long-term replayability.

Tech & dependencies

- **Tech:** Constraint-based generation (biome/theme variants, trap density, mutation cadence curves) with automated validation of the path-to-spawn invariant across many seeds, so every match is distinct yet always solvable.
- **Prerequisites:** M1G7 (procedural level generation).
- **Feeds into:** The replayability pillar; content variety at showcase.

Track Goals & Team Composition

Recommended composition: 6 CS + **2 Design** + 0 Art (8) — the proposal is **6 CS + 1 Design + 0 Art**, which is *accepted but tight*: with **no artist**, the designer must also pick up the **D11 conditional** (Art POC / Art Implementation Quality) each milestone, and a **solo designer + 0 artists** is exactly the case the design pool flags — such a team "must ask the instructor which conditional bends" (see [design-goals-pool.pdf](#)). Strongly advised: **add a 2nd designer** so one owns the game design (labyrinth mutation, loot/sabotage/countdown systems, level design) and the other absorbs the D11 art-surface conditional, keeping both reachable; the module rules allow it (min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the team stays at 1 designer, expect a lighter art surface — a **geometric / programmer-art** visual style (clean top-down shapes, colour-coded walls/traps/loot) that the cosmetic shop reskins — and the instructor sign-off on the bent conditional. *Scope note (advisory)*: a deterministic mutating-maze sim + a path-to-spawn-guaranteed generator + LAN lockstep + a direction-changing collision engine + a level editor is a **CS-heavy, well-focused** scope — the 2-player LAN match on one mutating map with basic loot/sabotage and the level editor is the M1/M2 floor; multiple map themes, the full trap/sabotage set, up-to-4-player support and mutation-pacing variety are the stretch.

Track goals — suggested Design / UI-UX / TEAM picks for this project (no Art track — 0 artists, so the Art surface is carried by the Design-owned D11 conditional); deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the mutating-labyrinth 2-player race, the loot/carry/deposit scoring, the sabotage mechanics and the timed-survival pressure; primary benchmark e.g. Ultimate Chicken Horse / SpeedRunners, with a Singapore benchmark for the local-awareness criterion)
- M1D2 Core Gameplay Loop Prototype (a greybox proving collect-loot → sabotage → escape under the countdown, with a mutating maze, is fun) · prototype in Unity
- M1D11 Art Proof of Concept *(conditional — mandatory for 0-artist teams; Design-owned)* — the geometric/programmer-art visual direction: palette, colour-coding for walls/traps/loot, and 2 mockup screens.

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D11 Art Implementation Quality *(conditional — mandatory for 0-artist teams; Design-owned)* — all in-game art implemented in-engine (no placeholders); the cosmetic-shop skins applied.

- **UI/UX M1** (graded by the Design instructor for the Games category):

- M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the countdown, the loot/carry/deposit indicators, the sabotage affordances and the LAN lobby.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — lobby/LAN-connect, in-match HUD, results and the cosmetic shop navigable unaided on the Native-Android target.
- M2U2 ★ UX Validation & Final Polished UI — validated, polished match + sabotage UX tested with players on the Pixel 9a.

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (tense chase BGM, loot pickup, trap trigger, wall-shift and countdown cues), suitable and correctly triggered.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for overlapping match SFX.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (chase BGM + loot / deposit / trap / wall-shift / sabotage / countdown SFX, ≥6 types), scaling with match intensity.
 - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; a distinct wall-shift + trap cue).
 - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the level editor + the network debug tool), prototype demo, team post-mortem (wk14)
-

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*